EE216 Reconfigurable Computing

Topic 10： SW & HW System Optimization

Prof. Yajun Ha

ShanghaiTech University
Shanghai, China

上海科技大学
ShanghaiTech University

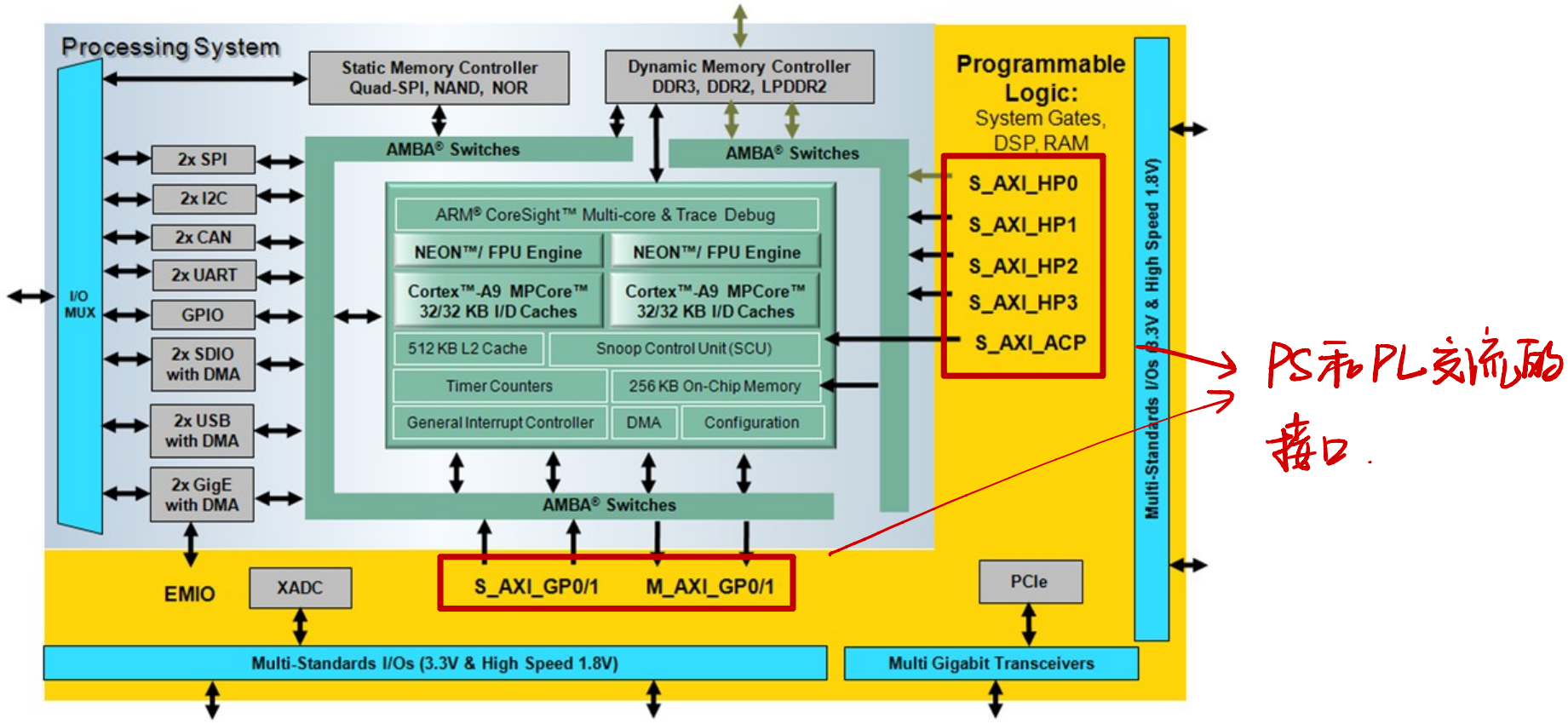
SW/HW System Optimization

- ☛ Data motion networks and optimization
- HLS optimization

Data Motion Network and Optimization

- PS-PL Interface AXI Ports
- Data Movement in Zynq
- Program Connectivity Implementation
- Compare Different Implementation

PS-PL Interface AXI Ports



- 2 general-purpose master ports
- 2 general-purpose slave ports
- 4 high-performance slave ports
- 1 accelerator coherency port (ACP) slave port

General-Purpose Master Ports

- Two identical 32-bit AXI master ports
  - M_AXI_GP0
  - M_AXI_GP1

- Attaches to slave AXI ports in programmable logic via PL AXI interconnect

- Mostly used for CPU and I/O peripheral block data movement to programmable logic

- Why two ports?
  - One for peripheral
  - One for SDSoC Control

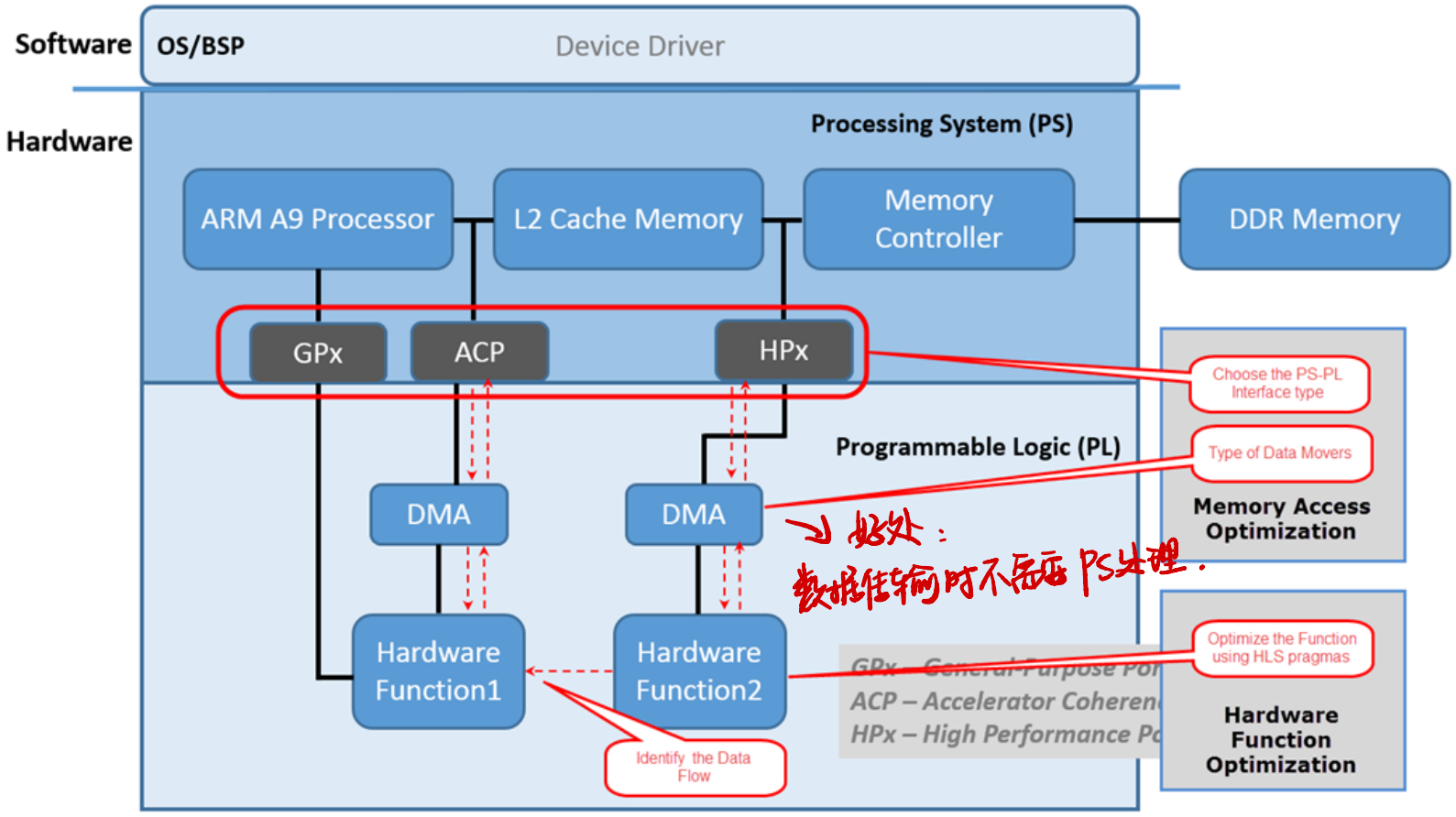
High-Performance Slave Ports

- Asynchronous crossing between the PL clock domain and PS clock domain
  - S_AXI_HP0, 1, 2, 3
- Provides initiator access from PL master to PS memory
- Provides low latency access to DDR and OCM
- Attaches to master AXI port in PL
  - DMA controller
  - MicroBlaze processor
  - Custom master built in programmable logic
  - Other third-party based IP

Accelerator Coherency Port (ACP) Slave Port

- One accelerator coherence port (ACP)
  - S_AXI_ACP
- High-performance, coprocessor interface to PL
- Provides low latency access to DDR and OCM
  - Accelerators gain access to the CPU cache hierarchy
- No drivers required for the ACP

Data Movement in Zynq



- Data may move between master/slave IP located in PL and memories located internal to the processor or external to the chip using the nine ports described in the previous section.
- The decision is made by the SDSoC compiler after analyzing the software or following user's directives

Program Connectivity Implementation

- Every connection with data movement between the software program and a hardware function requires a datamover
- Program connections can have multiple possible implementations
  - SDSoC compiler infers datamovers and system ports
  - Datamover and system ports can be overridden with pragma
- Datamover inference is based on payload size, payload memory attributes, and hardware function argument access pattern
  - #pragma SDS data data_mover(arg:<DM_TYPE>) // SIMPLE/SG
  - #pragma SDS data sys_port(arg:<Port_TYPE>) //ACP/HP/GP
  - #pragma SDS data copy/zero_copy(arg:<Length>)
  - #pragma SDS data access_pattern (A:SEQUENTIAL|RANDOM)

Program Connectivity Implementation

```
#define N 2048
#pragma SDS data copy(In[0:N],Out[0:N])
#pragma SDS data data_mover(In:AXIDMA_SG, Out:AXIDMA_SG)
#pragma SDS data access_pattern(In:SEQUENTIAL, Out:SEQUENTIAL)
#pragma SDS data sys_port(In:AFI, Out:AFI)
void IO(float* In, float* Out)
{
    float Buffer [N];
    for(int i=0;i<N;i++)   Buffer[i]=In[i];//Load data from DDR to PL
    for(int i=0;i<N;i++)   Out[i]=Buffer[i];//Write data from PL to DDR
}
```

Memory Models

- sds_lib.h functions to “declare” memory properties
  - sds_alloc(size_t size); // guaranteed physically continuous memory
  - malloc(size_t size); // physically uncontinuous(paged) memory

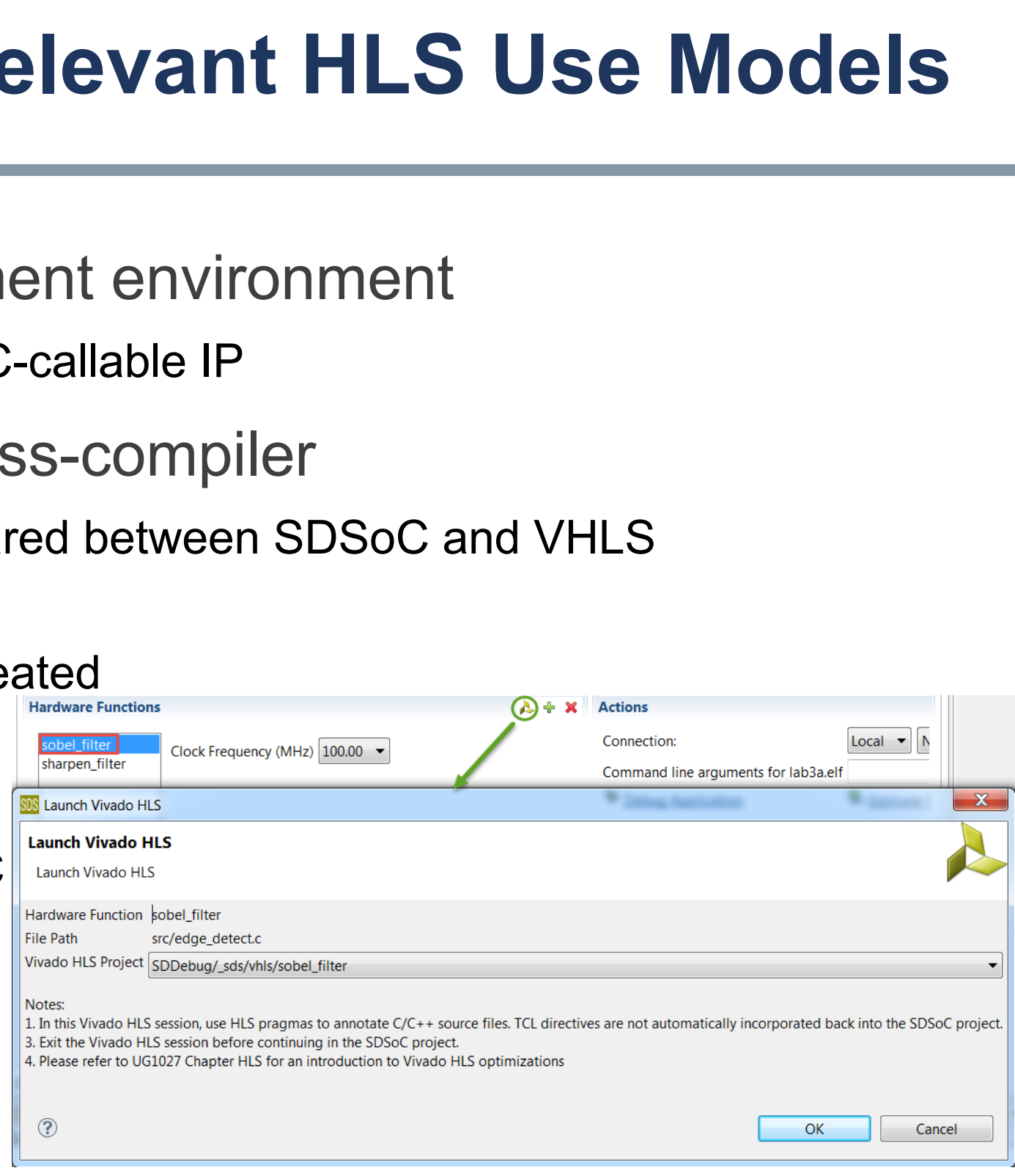
	Mem	Port	Datamover	copy	Setup	Transfer	Test	FF	LUT
1	Continuous	ACP	SIMPLE	zero	518	10890	48907	1197	1438
2	Continuous	AFI	SIMPLE	zero	11222	20675	110623	1197	1438
3	Continuous	ACP	SG	zero	518	10890	48891	1197	1438
4	Continuous	AFI	SG	zero	11222	20675	105411	1197	1438
5	Continuous	ACP	SIMPLE	copy	1120	10337	49912	130	161
6	Continuous	AFI	SIMPLE	copy	12454	10546	80345	130	161
7	Continuous	ACP	SG	copy	5202	15370	72418	130	161
8	Continuous	AFI	SG	copy	16242	15426	104536	130	161
9	Paged	ACP	SG	copy	28524	17532	321272	130	161
10	Paged	AFI	SG	copy	26624	18636	351497	130	161
11	Continuous	GP	FIFO	zero	518	10890	48941	1197	1438
12	Continuous	GP	FIFO	copy	396836	18636	580754	130	161
13	Paged	GP	FIFO	copy	396836	18636	573677	130	161

SW/HW System Optimization

- Data motion networks and optimization
- ☛ HLS optimization

Relevant HLS Use Models

- Vivado HLS as IP development environment
  - Export IP from HLS and import as C-callable IP
- Vivado HLS as Zynq PL cross-compiler
  - Hardware function source code shared between SDSoC and VHLS and ***the only*** handoff files
  - VHLS projects are automatically created by SDSoC and are temporary work products (no persistent state)
  - Optionally launch HLS from SDSoC
    - Optimize accelerator code
    - Simulate hardware function



Optimization Methodology

Baseline Hardware Function	• Determine performance with compiler defaults
1: Optimization for Metrics	• Define loop trip counts
2: Pipeline for Performance	• Pipeline and Dataflow
3: Optimize Structures for Performance	• Partition memories • Remove false dependencies
4: Reduce Latency	• Optionally specify latency requirements
5: Improve Area	• Optionally recover resources through sharing

Primary Optimization Techniques

- Memory access can be the biggest performance bottleneck
  - May have to restructure algorithm to optimize memory accesses for hardware (similar issues for GPU, DSPs)
  - Pointers are an efficient way to pass ownership to software functions, but not necessarily to hardware functions
  - Burst data into local arrays using memcpy or copy loops (e.g., line buffer, small matrices)
- Increase local concurrency
  - Pipeline and dataflow loops, function calls, and operations
  - Enhance pipeline performance

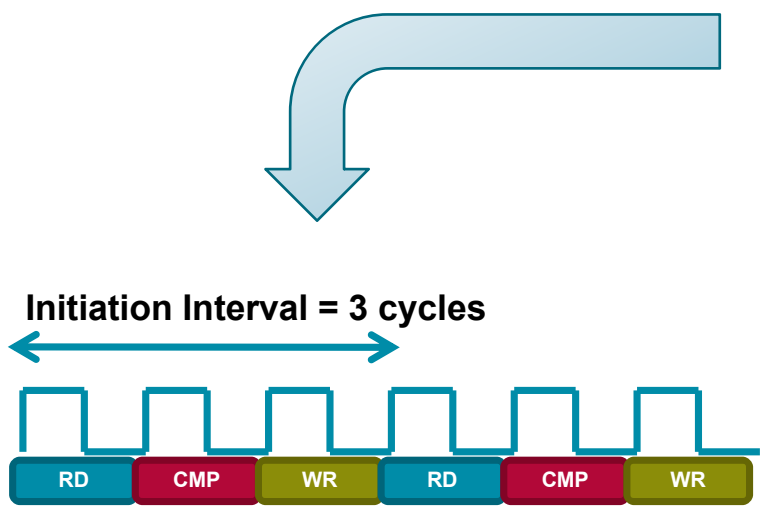
HLS Optimization Pragmas

Directives and Configurations	Description
PIPELINE	Reduces the initiation interval by allowing the concurrent execution of operations within a loop or function
DATAFLOW	Enables task level pipelining, allowing functions and loops to execute concurrently. Used to minimize interval
INLINE	Inlines a function, removing all function hierarchy. Used to enable logic optimization across function boundaries and improve latency/interval by reducing function call overhead
UNROLL	Unroll for-loops to create multiple independent operations rather than a single collection of operations
ARRAY_PARTITION	Partitions large arrays into multiple smaller arrays or into individual registers, to improve access to data and remove block RAM bottlenecks

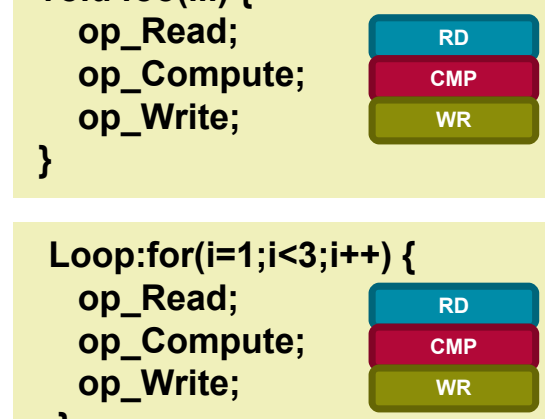
Loop and Function Pipelining

Without Pipelining

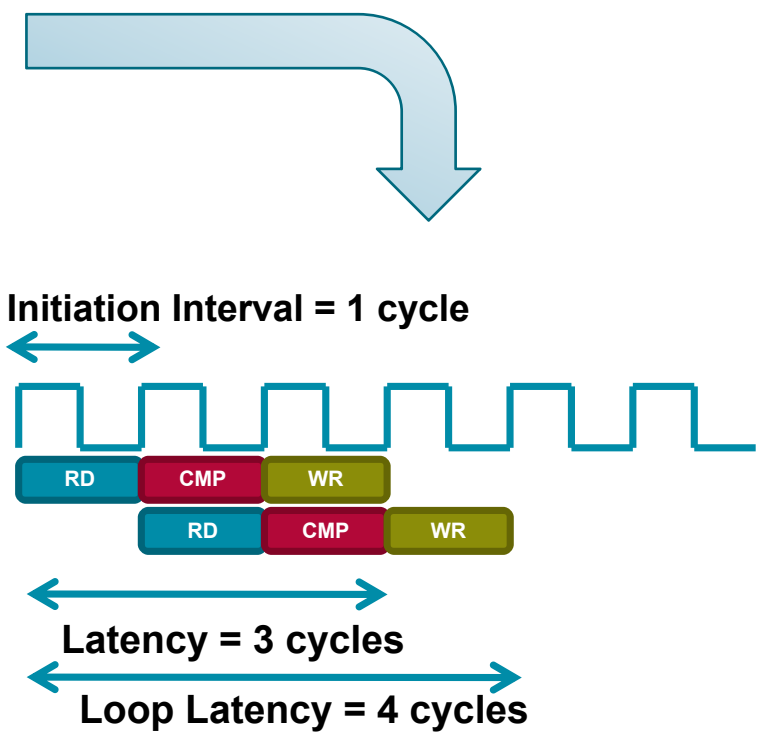
```
void foo(...) {
   op_Read;
   op_Compute;
   op_Write;
}
```




```
Loop:for(i=1;i<3;i++) {
   op_Read;
   op_Compute;
   op_Write;
 }
```



With Pipelining



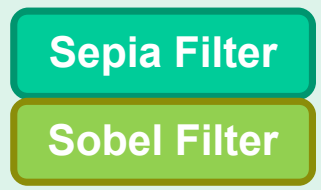

```
for (index_b = 0; index_b < B_NCOLS; index_b++) {
#pragma HLS PIPELINE II=1
     float result = 0;
     for (index_d = 0; index_d < A_NCOLS; index_d++) {
         float product_term = in_A[index_a][index_d] * in_B[index_d][index_b];
         result += product_term;
     }
     out_C[index_a * B_NCOLS + index_b] = result;
}
```

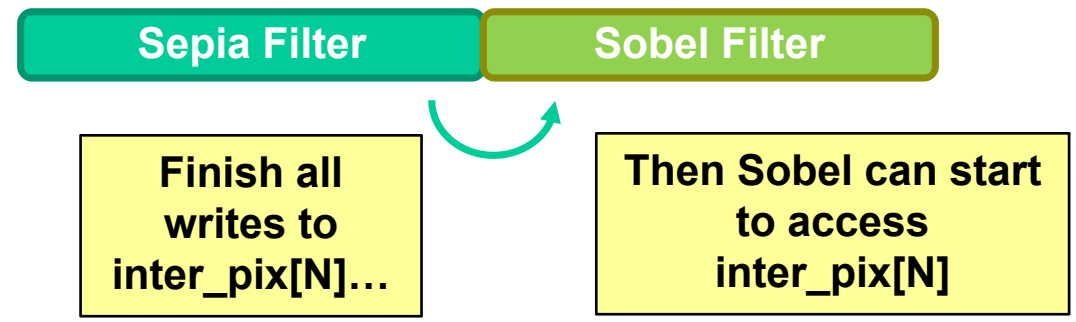
Dataflow

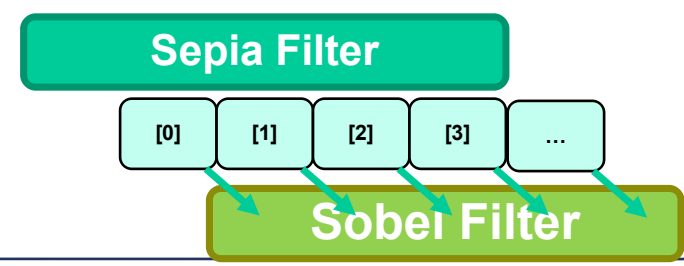
- Default behavior
  - Complete a function or loop iteration before starting next function or loop iteration
  - Datamover and system ports can be overridden with pragma
- Dataflow
  - Start next function or loop iteration as soon as “ready” and data is available
  - Initiation interval (II) represents number of clocks between ‘starts’
  - Increased concurrency
- Apply dataflow within a function but not at the top level

```
//This memory is turned into a FIFO during
optimization
  rgb_pixel inter_pix[MAX_HEIGHT][MAX_WIDTH];

 // Primary processing functions
  sepia_filter(in_pix,inter_pix);
  sobel_filter(inter_pix,out_pix2);
```

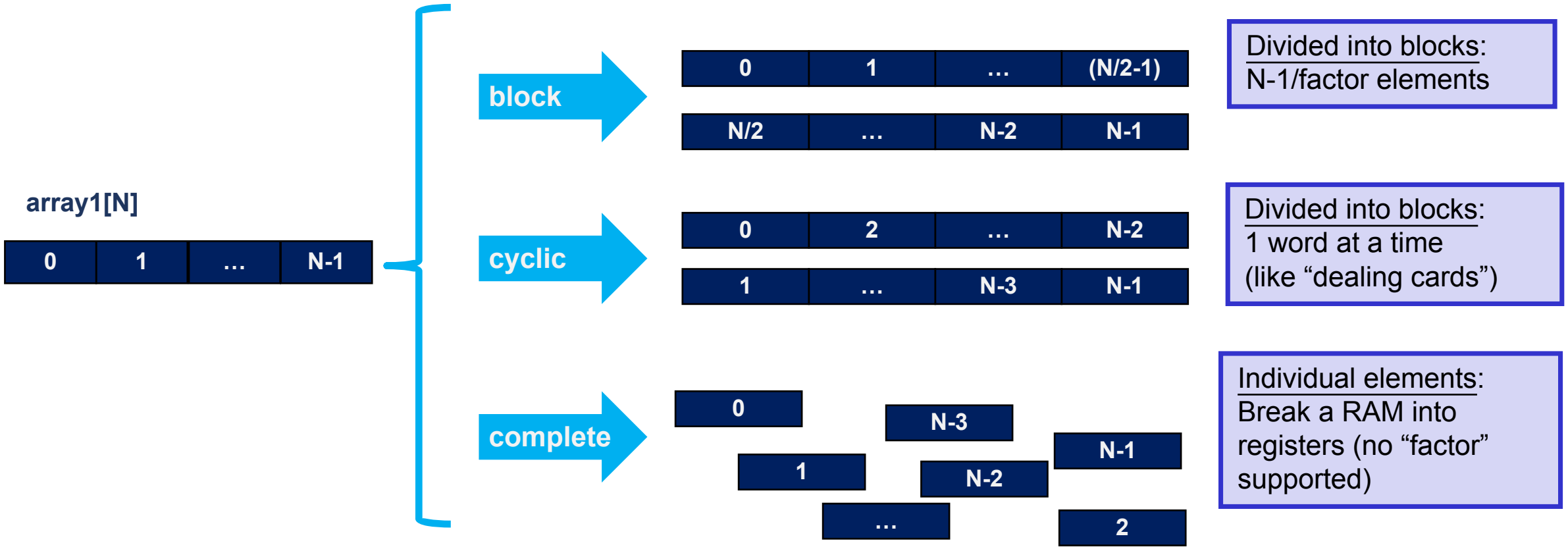






Array Partitioning

- Program connections can have multiple possible implementations




```
void mmult_kernel(float in_A[A_NROWS][A_NCOLS], float in_B[A_NCOLS][B_NCOLS], float out_C[A_NROWS*B_NCOLS])
{
#pragma HLS INLINE self
#pragma HLS array_partition variable=in_A block factor=16 dim=2
#pragma HLS array_partition variable=in_B block factor=16 dim=1
// snip
}
```

→ 把第2维分成16个block

→ 把第1维分成16个block.

SW/HW System Optimization

- Data motion networks and optimization
- HLS optimization